AI Job Application Agent

Small-Step, Phase-Wise Development Roadmap

Goal: Build the system gradually so that every phase produces a working piece. Do not jump directly to full auto-apply. Finish and test one small milestone before moving to the next.

Recommended rule: If a phase is not working reliably, do not start the next phase. Keep the project runnable after every phase.

Roadmap at a Glance

Phase	Main outcome
1–4	Project setup, FastAPI, database, configuration
5–8	Resume upload, extraction, structured profile
9–13	Job collection, normalization, storage, deduplication
14–17	AI matching, scoring, explanations, ranking
18–21	Frontend dashboard and job review
22–26	Playwright/browser automation foundation
27–31	Application form detection and profile autofill
32–35	AI application questions and grounded answers
36–39	Human approval and controlled submission
40–43	Application tracking, scheduling, notifications
44–47	Testing, security, reliability, deployment
48–50	Controlled auto-apply and final production polish

The roadmap contains **50 small phases**. A phase may take minutes or several hours depending on your experience. The important part is that each phase has a narrow objective and a clear completion test.

Part A — Foundation

Phase 1: Create the Project Folder

■ Create the main ai-job-agent folder.

■ Create backend, frontend, tests and workers folders.

■ Create README.md and .gitignore.

Done when: the folder structure exists and Git ignores secrets, virtual environments and generated files.

Phase 2: Create the Python Environment

■ Create a Python virtual environment.

■ Activate it.

■ Verify python and pip versions.

Done when: the virtual environment activates and Python runs correctly.

Phase 3: Install Backend Dependencies

■ Install FastAPI and Uvicorn.

■ Install Pydantic and SQLAlchemy.

■ Install environment/configuration helpers.

Done when: pip install completes and imports work without errors.

Phase 4: Create the First FastAPI Server

■ Create app/main.py.

■ Add a /health endpoint.

■ Run the server with Uvicorn.

Done when: opening /health returns a successful response.

Part B — Resume & Candidate Profile

Phase 5: Create Resume Upload Endpoint

■ Create POST /resume/upload.

■ Accept PDF and DOCX files.

■ Reject unsupported extensions.

Done when: a test resume can be uploaded through the API.

Phase 6: Extract PDF Text

■ Install PyMuPDF.

■ Read uploaded PDF pages.

■ Return extracted text.

Done when: a normal text-based PDF produces readable text.

Phase 7: Extract DOCX Text

■ Install python-docx.

■ Read paragraphs from DOCX.

■ Return extracted text.

Done when: a DOCX resume produces readable text.

Phase 8: Create Candidate Profile JSON

■ Define fields for name, skills, education, experience and projects.

■ Add target roles, locations and preferences.

■ Save the parsed profile.

Done when: the resume can be converted into a structured candidate profile.

Part C — Job Collection

Phase 9: Create the Job Database Model

■ Create the Job model.

■ Add title, company, location, description and URL.

■ Add source and external identifier fields.

Done when: a job can be stored and retrieved from PostgreSQL.

Phase 10: Connect PostgreSQL

■ Create the database connection.

■ Create environment variables for credentials.

■ Run the first database migration.

Done when: the API can connect to PostgreSQL without hardcoded credentials.

Phase 11: Build the First Job Source Adapter

■ Choose one permitted/reliable source.

■ Write a small adapter that returns jobs.

■ Convert its result to your internal Job schema.

Done when: at least one source returns normalized job objects.

Phase 12: Store Collected Jobs

■ Send normalized jobs to PostgreSQL.

■ Avoid storing the same external job twice.

■ Add created_at and updated_at timestamps.

Done when: jobs collected by the adapter appear in the database.

Phase 13: Add Job Filtering & Deduplication

■ Filter by role keywords.

■ Filter by location and experience when available.

■ Deduplicate by source + external ID or canonical URL.

Done when: the system produces a clean set of relevant, unique jobs.

Part D — AI Job Matching

Phase 14: Create a Basic Rule-Based Matcher

■ Compare candidate skills with job skills/keywords.

■ Compare target role with job title.

■ Calculate a simple baseline score.

Done when: the system produces a deterministic match score before using an LLM.

Phase 15: Add Semantic Matching

■ Create embeddings for the candidate profile and job descriptions.

■ Store vectors using pgvector or another vector store.

■ Calculate semantic similarity.

Done when: semantically similar jobs rank above clearly unrelated jobs.

Phase 16: Add LLM Match Reasoning

■ Send candidate profile and job description to the LLM.

■ Ask for strengths, gaps and role fit.

■ Return structured JSON rather than free-form text only.

Done when: each strong job has an explanation grounded in the available information.

Phase 17: Create the Final Match Score

■ Combine rules, semantic similarity and LLM reasoning.

■ Use the planned weights: skills, experience, role, location, education and responsibilities.

■ Store score and explanation in job_matches.

Done when: every job has a consistent score and explanation.

Part E — Dashboard

Phase 18: Create the React/Next.js App

■ Create the frontend project.

■ Create a basic layout.

■ Connect it to the FastAPI backend.

Done when: the frontend loads and can call the backend.

Phase 19: Build the Resume/Profile Screen

■ Show parsed resume information.

■ Allow target roles and locations to be edited.

■ Add minimum match score preference.

Done when: the user can view and update their profile.

Phase 20: Build the Job List Screen

■ Show title, company, location and match score.

■ Add sorting by match score.

■ Add filters for role, location and source.

Done when: the user can browse and filter matched jobs.

Phase 21: Build the Job Detail Screen

■ Show full job description.

■ Show match explanation and skill gaps.

■ Show the original application link.

Done when: a user can make an informed decision about a job before applying.

Part F — Browser Automation Foundation

Phase 22: Install Playwright

■ Install Playwright for Python.

■ Install the required browser.

■ Run a simple test page.

Done when: Playwright can launch a browser successfully.

Phase 23: Create a Browser Service

■ Create browser.py.

■ Centralize browser launch and shutdown.

■ Add basic timeout and logging configuration.

Done when: browser lifecycle is managed from one place.

Phase 24: Open a Job Application Page

■ Pass an application URL to Playwright.

■ Open the page.

■ Capture page title and URL.

Done when: the agent can reliably open a selected application page.

Phase 25: Inspect the Page Structure

■ Read visible text and form controls.

■ Identify inputs, selects, textareas and file inputs.

■ Save a diagnostic representation for debugging.

Done when: you can see what fields the automation agent is actually working with.

Phase 26: Add a Safe Browser Session

■ Keep session state isolated.

■ Do not hardcode passwords.

■ Add a manual login path where required.

Done when: browser sessions can be used without exposing credentials in source code.

Part G — Application Form Automation

Phase 27: Detect Common Form Fields

■ Detect first name, last name, email and phone.

■ Use labels, names, IDs and accessible attributes.

■ Create a field-mapping layer.

Done when: common fields are detected without relying on screen coordinates.

Phase 28: Connect Fields to Candidate Data

■ Map detected fields to the candidate profile.

■ Fill only fields with known values.

■ Log which fields were filled.

Done when: a test form can be filled from the candidate profile.

Phase 29: Handle Resume Uploads

■ Detect file inputs.

■ Select the correct resume version.

■ Verify that the file is attached.

Done when: the correct resume is attached to a test application.

Phase 30: Handle Selects and Checkboxes

■ Support dropdowns.

■ Support radio buttons and checkboxes.

■ Flag ambiguous choices instead of guessing.

Done when: standard non-text form controls are handled safely.

Phase 31: Create an Application Draft

■ Open a job.

■ Fill known fields.

■ Stop before submission.

Done when: the agent can prepare an application without submitting it.

Part H — AI Application Intelligence

Phase 32: Create the Question Extraction Step

■ Collect application questions.

■ Separate standard profile fields from open-ended questions.

■ Mark required vs optional questions.

Done when: open-ended questions are identified for AI processing or human review.

Phase 33: Create the Grounded Question Agent

■ Provide resume, candidate profile and job description.

■ Require structured answers.

■ Forbid invented experience or qualifications.

Done when: generated answers only use supported candidate information.

Phase 34: Add Resume Tailoring

■ Create a job-specific resume draft.

■ Preserve factual information.

■ Track the resume version used for each application.

Done when: a tailored resume can be generated without fabricating credentials.

Phase 35: Add Cover Letter Generation

■ Generate a concise role-specific cover letter.

■ Ground it in the candidate profile and job description.

■ Allow editing before use.

Done when: a draft cover letter is generated and editable.

Part I — Human Approval & Submission

Phase 36: Create Application Review UI

■ Show the job and match score.

■ Show every proposed field value.

■ Show AI-generated answers.

Done when: the user can inspect the complete application before submission.

Phase 37: Add Edit & Approve Actions

■ Allow editing generated answers.

■ Allow individual fields to be corrected.

■ Add an Approve button.

Done when: the user can modify the application and explicitly approve it.

Phase 38: Add Final Submission Step

■ Only approved applications can reach submission.

■ Submit through the browser workflow.

■ Capture the result and any confirmation information.

Done when: an approved test application can be submitted and its outcome recorded.

Phase 39: Add Failure Recovery

■ Detect validation errors.

■ Save the current state.

■ Stop safely instead of repeatedly submitting.

Done when: a failed application does not result in uncontrolled retries or duplicate submissions.

Part J — Tracking & Automation

Phase 40: Create Application Statuses

■ Add SAVED, MATCHED, READY, APPROVED and SUBMITTED.

■ Add REJECTED, INTERVIEW, OFFER and WITHDRAWN.

■ Store status history where useful.

Done when: every application has a clear lifecycle.

Phase 41: Build Application Tracker

■ Create an applications page.

■ Filter by status.

■ Show dates, company, role and source.

Done when: the user can see all applications in one place.

Phase 42: Add Background Jobs

■ Add Redis and Celery or an equivalent worker.

■ Move long-running job searches outside the request cycle.

■ Log task status.

Done when: job collection can run asynchronously.

Phase 43: Add Scheduled Job Discovery

■ Create a daily search task.

■ Search configured sources.

■ Save only new/relevant jobs.

Done when: the system can discover jobs on a schedule without manual triggering.

Part K — Quality, Security & Deployment

Phase 44: Add Unit Tests

■ Test resume parsing.

■ Test job normalization.

■ Test scoring and deduplication.

Done when: core logic has repeatable automated tests.

Phase 45: Add Browser Integration Tests

■ Test field detection.

■ Test autofill on controlled test pages.

■ Test failure handling.

Done when: browser automation can be regression-tested.

Phase 46: Harden Security

■ Move secrets to environment variables or a secret manager.

■ Encrypt sensitive stored data where appropriate.

■ Never commit credentials or session secrets.

Done when: security-sensitive configuration is separated from source code.

Phase 47: Dockerize & Deploy

■ Create Dockerfiles.

■ Create docker-compose for local development.

■ Deploy backend, database, worker and frontend.

Done when: the complete system can be started reproducibly.

Part L — Controlled Auto-Apply

Phase 48: Create Auto-Apply Rules

■ Set a minimum match score.

■ Restrict target roles and locations.

■ Exclude jobs already applied to.

Done when: the system has explicit, reviewable rules for what it may process automatically.

Phase 49: Enable Limited Auto-Apply

■ Start with one reliable application flow.

■ Keep human approval for unusual/high-risk questions.

■ Add rate limits and duplicate-submission protection.

Done when: the system can submit only applications that satisfy the configured rules and safety checks.

Phase 50: Production Monitoring & Polish

■ Track success/failure rates.

■ Add logs and alerts for broken selectors or source changes.

■ Improve UI, documentation and onboarding.

■ Document supported sources and known limitations.

Done when: the project is stable enough to use repeatedly and maintain over time.

Final Architecture After Phase 50

Layer	Responsibility
Frontend	Profile, jobs, match explanations, application review, tracker
FastAPI	API endpoints and orchestration
AI layer	Resume parsing, matching, Q&A, tailoring
Job adapters	Source-specific discovery and normalization
Playwright	Controlled browser interaction
PostgreSQL + pgvector	Application data and semantic search
Redis + worker	Background and scheduled tasks
Security	Secrets, sessions, access controls and audit logging

Final Daily Workflow

→ Scheduled search starts.

→ Job sources return new postings.

→ Jobs are normalized and deduplicated.

→ Candidate profile and jobs are matched.

→ High-match jobs appear in the dashboard.

→ Application Agent prepares selected applications.

→ AI generates grounded answers for open-ended questions.

→ User reviews and approves applications according to the configured mode.

→ Approved applications are submitted through supported workflows.

→ Application status and evidence are stored.

Important Development Rule

Do not build the whole agent at once. Finish Phase 1, test it, commit it to Git, then move to Phase 2. At every phase, keep the system in a runnable state. This will make debugging dramatically easier and will let you identify whether a failure comes from the AI layer, database, browser automation, job source, or frontend.

Suggested first milestone: Complete Phases 1–8 only. At that point you should have a working backend that accepts a resume and produces a structured candidate profile. That becomes the foundation for everything else.